

```
    quicksort(items, left, i - 1)    # RECURSIVE CASE
    quicksort(items, i + 1, right)  # RECURSIVE CASE

myList = [0, 7, 6, 3, 1, 2, 5, 4]
quicksort(myList)
print(myList)
```

The *quicksort.html* program contains the JavaScript equivalent:

```
<script type="text/javascript">
function quicksort(items, left, right) {
    // By default, `left` and `right` span the entire range of `items`:
    if (left === undefined) {
        left = 0; // `left` defaults to the 0 index.
    }
    if (right === undefined) {
        right = items.length - 1; // `right` defaults to the last index.
    }

    document.write("<br /><pre>quicksort() called on this range: [" +
        items.slice(left, right + 1).join(", ") + "]</pre>");
    document.write("<pre>.....The full list is: [" + items.join(", ")
+ "]"</pre>");

    if (right <= left) { ❶
        // With only zero or one item, `items` is already sorted.
        return; // BASE CASE
    }

    // START OF THE PARTITIONING
    let i = left; ❷ // i starts at the left end of the range.
    let pivotValue = items[right]; // Select the last value for the pivot.

    document.write("<pre>.....The pivot is: " + pivotValue.toStrin
g() +
"</pre>");

    // Iterate up to, but not including, the pivot:
    for (let j = left; j < right; j++) {
        // If a value is less than the pivot, swap it so that it's on the
        // left side of `items`:
        if (items[j] <= pivotValue) {
            // Swap these two values:
            [items[i], items[j]] = [items[j], items[i]]; ❸
```